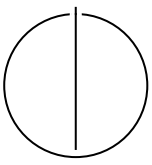
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Author



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

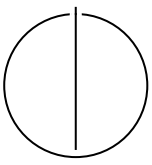
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Titel der Abschlussarbeit

Author:	Author
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



Acknowledgments

Abstract

Contents

1 Introduction

1.1 Motivation

Travel destination selection is a complex, multi criteria process in which users weigh explicit parameters, such as budget and seasonality, against subjective preferences, such as atmosphere and climate [Die+22]. Conventional recommendation systems often rely on one-shot interaction patterns or predefined interface controls, which can limit how precisely users articulate these nuanced needs [Jan22]. Consequently, the resulting recommendations may reflect the structure of the interface more strongly than the user's actual intent.

Natural language offers an effective response to this limitation. As an intuitive input modality, it enables richer and more flexible expression of information needs without forcing users to adapt to a system's predefined interaction logic [AGZ24]. A chat based design therefore lowers the entry barrier and supports progressive preference refinement over time. This iterative process is particularly relevant for travel planning, because users may initially be unaware of their own needs or unable to express them precisely, especially in the early stages of decision making [AH22].

However, relying exclusively on conversational interfaces introduces new usability challenges. While they offer high input flexibility, these systems typically present recommendations as dense, unstructured text. Confronted with large amounts of text, users often struggle to grasp the main points and properly interpret the proposed recommendations.

Integrating a graphical presentation layer mitigates these comprehension barriers. Visual representations of destinations make recommendation outcomes immediately inspectable and easier to rank. By translating dense textual explanations into an accessible visual format, graphical elements perfectly complement conversation, reducing cognitive load and improving the overall clarity of the suggestions.

Ultimately, there is a clear need for a travel recommendation system that frees users from the limitations of predefined search criteria while presenting results in an easy to understand format. The core challenge lies in bridging the gap between the expressive freedom of open-ended input and the structural clarity required to evaluate complex options. This thesis is motivated by this tension, exploring how to design interaction models that allow users to naturally articulate their needs without being overwhelmed

by the effort required to interpret the outcome.

1.2 Problem Statement

The central problem addressed in this thesis is how to derive reasonable travel destination recommendations from natural language user queries while keeping the recommendation process grounded in available travel destination data. Users naturally describe travel needs in open-ended language, but the system must convert these expressions into recommendation decisions that can be supported by concrete destination information. The problem therefore lies in bridging the gap between flexible human communication and the structured representations required for reliable recommendation.

This problem is not trivial because natural language queries often combine explicit constraints with qualitative preferences. A user may specify factual conditions, such as budget, season, or travel duration, together with more subjective expectations regarding activities, atmosphere, or the general character of a destination. These requirements are not always expressed in a complete or directly machine interpretable form. Consequently, the system must identify relevant preferences, determine which parts of the request can be mapped to structured data, and use the available destination information to generate recommendations that remain plausible and justified.

At the same time, data grounding imposes an important constraint on the recommendation process. A conversational interface may allow users to formulate preferences freely, but recommendation quality depends on whether those preferences can be matched to actual destination records rather than answered through unsupported language generation alone. The system must therefore transform natural language input into actionable retrieval and filtering operations while preserving enough meaning from the original request to produce useful results. In addition, because travel recommendation is often iterative, the system should maintain contextual continuity across multiple turns when users refine or partially revise earlier preferences.

Based on this problem formulation, this thesis investigates how a conversational recommendation pipeline and a hybrid user interface can be combined to support natural language travel queries and produce recommendations that remain both understandable to the user and grounded in destination data.

1.3 Research Objectives

The thesis is guided by the following research objectives:

1. Investigate techniques for extracting user intents and travel preferences from unstructured natural language queries and mapping them to structured travel data to generate data-grounded recommendations.
2. Examine how a hybrid user interface can bridge agentic AI reasoning and user interaction by presenting intermediate decision making processes and recommendation outcomes in an interpretable format.
3. Investigate mechanisms for maintaining contextual awareness across multi-turn conversations, enabling the continuous refinement of travel recommendations through natural language interactions.
4. Design and develop a prototype travel destination recommendation system based on a hybrid user interface and agentic AI, incorporating intent extraction and data-grounded recommendations.

1.4 Thesis Structure TODO

Chapter 2 introduces the conceptual foundations required for the thesis, including recommendation systems, large language models, agentic artificial intelligence, and hybrid interfaces. Chapter 3 positions the thesis relative to existing work on travel recommendation systems and agentic recommendation approaches.

Chapter 5 presents the architecture of the proposed prototype, covering the backend, frontend, recommendation workflow, and communication between components. Chapter 6 defines the evaluation strategy, which combines a synthetic assessment of recommendation quality and conversational memory with a user survey on interface quality and perceived usefulness. Finally, Chapter 7 summarizes the contribution of the thesis and outlines limitations and directions for future work.

2 Background

2.1 Recommendation Systems

A recommendation system is an information system that estimates which items from a set of available options are most relevant for a particular user and presents these items as suggestions [AT05]. Its purpose is to support decision making by reducing the effort required to search through many alternatives and by prioritizing those that are most likely to match the user's needs, interests, or goals.

To do so, a recommendation system uses a representation of the user and a representation of the available items. User information may include explicit preferences, prior behavior, or contextual constraints. Item information may include structured attributes, textual descriptions, or metadata. Based on these inputs, the system identifies relevant candidates, ranks them, and presents the result to the user [Lü+12]. Recommendation is therefore not only a retrieval task, but also a relevance estimation task.

2.1.1 Main Recommendation Paradigms

Several recommendation paradigms are commonly distinguished, each relying on a different source of evidence for estimating relevance [AT05].

Collaborative Filtering. Collaborative filtering infers preferences from interaction patterns across many users and items. It is effective when sufficient behavioral data is available, but it is less suitable when user histories are sparse or when preferences must be interpreted in a highly specific way [SK09].

Content-Based Recommendation. Content-based recommendation relies on the attributes of the items themselves. It recommends items whose properties are similar to a user's known or stated preferences. Its effectiveness depends strongly on the quality and completeness of the item descriptions [AT05].

Knowledge-Based Recommendation. Knowledge-based recommendation uses explicit constraints, domain knowledge, or decision rules to connect user requirements

with item properties. This is particularly relevant in domains where decisions are complex and cannot be inferred reliably from interaction history alone [AT05].

Hybrid Recommendation. Hybrid recommendation combines multiple paradigms in order to compensate for the limitations of any single method. In practice, such combinations are common because user needs, item representations, and available data are heterogeneous [Bur02].

2.1.2 Travel Recommendation as a Special Case

Travel recommendation is especially challenging because destination choice depends on a combination of explicit constraints and subjective preferences [Die+22]. Users may specify budget, season, or travel duration, while also expressing qualitative expectations related to climate, activities, atmosphere, or the intended type of experience [AH22]. These requirements are often incomplete, ambiguous, or difficult to reduce to a fixed set of attributes.

In addition, historical interaction data can be less informative in travel recommendation than in domains with denser and more repetitive feedback. Destinations are also complex items that must be described through both structured and descriptive information. As a result, travel recommendation often requires a combination of filtering, retrieval, and richer preference interpretation [Die+22].

2.2 Large Language Models

Large language models are neural language processing systems trained on very large text corpora to predict likely continuations of text sequences [Zha+26]. Through this objective, they learn statistical regularities of language at multiple levels, including syntax, semantic associations, discourse structure, and recurring relations between instructions and appropriate responses. In practical use, this makes them capable of generating coherent text, following prompts, and adapting their output to the immediate conversational context.

Their importance is not limited to text generation alone. Because they process input in context, large language models can derive structured interpretations from free form language. They can identify relevant entities, extract explicit constraints, relate several conditions within one utterance, resolve references to earlier statements, and reformulate vague or underspecified requests into clearer representations [Zha+26]. They can also summarize longer inputs, classify intent, transform unstructured descriptions into more regular outputs, and maintain local coherence across multiple dialogue turns.

These operations should not be understood as reliable human like reasoning, but they do amount to a useful form of language-based inference.

These capabilities are especially relevant in settings where users communicate through natural language rather than fixed forms or filters. In such cases, the model can help bridge the gap between the flexible way users express goals and the more explicit representations that software systems typically require. For this reason, large language models are attractive components in applications that need to interpret open-ended requests, preserve context, and produce responses that remain readable and conversational [AGZ24].

An important problem that occurs in large language models is hallucination, meaning that the model can produce statements that are fluent and plausible in form, but unsupported, inaccurate, or entirely fabricated with respect to the underlying facts, the provided input, or the available evidence [Zha+26; Gao+23]. This problem follows from the training objective: the model is optimized to predict likely continuations of text, not to verify whether a generated statement is true in a specific application context. As a result, linguistic coherence can create an impression of reliability even when the content is wrong. Hallucinations can therefore take several forms, including fabricated facts, incorrect attributions, non existent references, or claims that go beyond what the input actually supports.

In recommendation systems, such models can in principle be used in two different ways. First, they can be used as standalone recommenders that directly transform a user request into suggested items and explanations. This offers high flexibility, but it is especially vulnerable to hallucinations, unsupported claims, and recommendations that are only loosely tied to the available data [Gao+23]. Second, they can be used as components within a broader recommendation pipeline, where they support tasks such as request interpretation, constraint extraction, and response formulation, while retrieval and candidate selection remain tied to explicit data representations. This reduces the freedom of the model to invent unsupported content, because crucial decisions remain connected to explicit and inspectable data. More generally, this distinction points to a broader limitation of natural language output: even when text is coherent and interpretable to humans, software systems still require a computational representation of meaning that can be compared, searched, and matched systematically. One important approach to this problem is the use of embeddings.

2.3 Embeddings

Embeddings are dense numerical vectors that represent words, phrases, or longer texts in a fixed dimensional form [Mik+13]. Instead of describing meaning through

manually defined categories, an embedding model assigns each input a position in a high dimensional vector space. The basic idea is simple: texts that occur in similar contexts or express similar meanings should receive similar vector representations. As a result, semantically related items tend to lie closer to each other in vector space than unrelated ones.

The generation of embeddings depends on the model and training objective. In early distributed representation approaches, vector values are learned from co-occurrence patterns in large corpora, so that words used in similar linguistic environments obtain similar representations [Mik+13]. More recent encoder-based methods extend this idea to whole sentences or short documents. In these models, the input text is processed by a neural encoder and mapped to a single vector that captures information about its overall meaning [RG19]. Once such vectors are available, texts can be compared computationally with similarity measures such as cosine similarity, which estimates how closely two vectors point in the same direction.

Within this thesis, embeddings provide a bridge between natural language preferences and destination descriptions. A user's request may refer to "quiet coastal towns" while the underlying data describes "relaxed seaside destinations" or "small beach cities." Exact keyword matching can miss such correspondences, whereas sentence level embeddings support approximate semantic retrieval over descriptive text fields [RG19]. At the same time, embeddings provide only a similarity signal. They do not by themselves determine which constraints are mandatory, how conflicting preferences should be resolved, or how recommendation results should be explained. They are therefore best understood as one retrieval-oriented component within a larger system, not as a complete decision mechanism on their own. Coordinating these operations requires a higher level workflow, which motivates the agentic perspective discussed next.

2.4 Agentic Artificial Intelligence

A practical definition of an AI agent describes it as a system that uses a large language model to decide the control flow of an application [Cha24]. In this view, the language model is not limited to producing a final response. It can also determine which step should follow next, whether additional information is needed, whether an external tool should be invoked, or whether processing should continue iteratively. Agentic artificial intelligence therefore refers to application designs in which the language model takes an active role in coordinating behavior rather than serving only as a passive text generator.

Agentic behavior can vary in degree. At a relatively simple level, a model may act as a router that selects between predefined branches. In more advanced settings, it may repeatedly evaluate intermediate results, call tools, maintain state, revise its course

of action, and continue operating until a stopping condition is reached [Cha24]. The main idea is that the model does not only generate content, it also influences how the application proceeds. This introduces a level of adaptive behavior that is difficult to achieve with fixed, purely linear prompt-response pipelines.

This is useful when a task is inherently multi step, when later actions depend on earlier results, or when the system must decide dynamically how much work is still required. Agentic designs can support decomposition of complex problems into smaller operations, selective tool use, interaction with external data sources, iterative refinement, and explicit handling of intermediate state. They are therefore attractive in settings where a system must not only answer a question, but also search, compare, verify, revise, or coordinate several operations before producing an output. At the same time, these benefits come with trade offs: agentic systems are more difficult to design, evaluate, and monitor, and additional orchestration steps create more opportunities for error propagation.

This architectural style can also be applied to recommendation scenarios. Instead of treating recommendation as a single generation step, an agentic system can separate tasks such as request interpretation, constraint extraction, retrieval, ranking, and response synthesis. This can be useful when the recommendation result is not bounded to a single query, but can instead be refined incrementally over multiple steps.

2.5 Hybrid Interfaces

Hybrid interfaces combine conversational interaction with graphical or structured user interface elements within a single workflow [Jär25]. Rather than forcing users to choose between chat and a conventional interface, a hybrid design treats both as complementary interaction modes. Natural language can be used to express open-ended goals, clarifications, or revisions, while structured components can externalize selections, constraints, intermediate states, and results in a more compact and inspectable form [Jan22].

Such an approach is relevant because conversational interaction and graphical interaction solve different problems. Natural language is effective for expressing nuanced preferences, incomplete ideas, and iterative revisions. Graphical components are better suited for comparing alternatives, exposing filters and constraints, presenting recommendation evidence, and supporting direct user control. A hybrid interface therefore aims to combine flexibility with interpretability rather than privileging one interaction style at the expense of the other [CP12].

This becomes especially useful in the context of agentic AI. If a system interprets a request, retrieves information, evaluates alternatives, and refines results over several

steps, then the interaction should not be limited to a final text response alone. Users may benefit from seeing which constraints are currently active, which alternatives are under consideration, what intermediate results have been retrieved, and where they can intervene to redirect the process [Cha24]. Hybrid interfaces can support this by connecting conversational exchange with explicit visual feedback and controllable interface elements.

In this way, a hybrid interface can make an agentic system more understandable and easier to steer. It can reduce the opacity that arises when multi step processing is compressed into a single textual answer, and it can help users correct misunderstandings before they propagate through later steps. At the same time, hybrid designs introduce their own challenges: they must coordinate multiple interaction modes without overloading the user, and they must decide carefully which internal states should be exposed and which should remain hidden. When designed well, however, they provide an effective interaction layer for applications that combine conversational flexibility with structured, multi step decision support.

3 Related Work

This chapter positions the thesis relative to existing work on travel recommendation systems and recent agentic approaches. The reviewed systems are grouped into interface-oriented travel recommenders and approaches that explicitly incorporate language-model-based reasoning or orchestration.

3.1 Travel Recommendation Systems with Classical Interfaces

Existing travel recommendation systems with classical interfaces provide an important baseline for this thesis because they inherit many of their interaction principles from recommender systems more broadly. In the general recommender-systems literature, recommendation has long been operationalized through structured user input, explicit preference models, ranked result lists, filtering controls, and inspectable item representations rather than through open-ended conversation [AT05]. This emphasis on usable interaction is also reflected in work that frames recommender systems not only as algorithmic ranking tools but as systems whose value depends on the user experience they provide [KR12]. Travel recommendation adopted these same design principles and adapted them to tourism-specific decision tasks. Until the recent spread of large-language-model-based conversational systems, the field was therefore naturally dominated by classical interfaces such as search forms, faceted filters, maps, ranked lists, and stepwise preference elicitation dialogs. A broad survey of tourism recommender systems likewise describes the area primarily through these graphical and form-based interaction patterns and the recommendation pipelines behind them [BMV14]. This historical dominance followed from the structure of pre-LLM recommender systems: users had to express their needs through explicit attributes or selectable constraints, the system transformed these inputs into recommendation logic, and the resulting candidates were returned in inspectable views that supported comparison and refinement [Lü+12].

At the algorithmic level, both general recommender systems and travel-specific systems have typically relied on collaborative filtering, content-based recommendation, knowledge-based reasoning, or hybrid combinations of these paradigms [Bur02]. Collaborative filtering in particular became one of the foundational paradigms of recommendation, but its effectiveness depends on the availability of suitable interaction data [SK09]. In the travel domain, knowledge-based and hybrid approaches have been

especially relevant because destination choice depends on sparse historical feedback, hard constraints such as budget or season, and subjective preferences that are difficult to infer reliably from behavior alone [CT20]. Classical interfaces complement these approaches by making constraints explicit and by enabling iterative preference refinement through filtering, critiquing, and direct inspection of alternatives [CP12]. Their main strength is transparency and user control. Their main limitation is that users must adapt nuanced travel goals to the structure of the available controls.

The Destination Finder application illustrates a lightweight form of this interaction style. It centers destination exploration around direct user input and interactive browsing rather than open-ended conversational interpretation []. This type of interface keeps the search space visible and easy to manipulate, but it does not inherently resolve the challenge of interpreting complex, open-ended travel requests.

DestiRec investigates user control in complex recommender systems [Sar25]. From the perspective of the present thesis, this line of work is relevant because it frames recommendation not only as an accuracy problem but also as an interaction problem. In travel recommendation, this distinction matters: even technically appropriate suggestions may be less useful if users cannot understand, influence, or iteratively refine the recommendation process. Sarica's thesis is also related to the present work through its use of map-based visualization and a destination dataset in a travel recommendation setting.

3.2 Agentic and Language-Model-Based Travel Recommendation Approaches

Recent work no longer treats language models only as generators of final recommendation text. Instead, they are increasingly used as orchestration components that interpret requests, select tools, trigger retrieval, and coordinate downstream recommendation steps. Peng et al. survey this development at the recommender-systems level and describe LLM-powered agents as a shift from static pipelines toward systems that can plan and adapt across multiple steps [Pen+25]. This perspective is directly relevant to the present thesis because the proposed prototype also separates interpretation, retrieval, and response generation rather than relying on one monolithic prompt.

In travel recommendation, this shift is visible in work that combines conversational preference elicitation with data access and explanation. Chen presents an agentic trip-planning application in which planning is structured as a workflow rather than a one-shot answer [Che]. Tandon and Banerjee study intent classification and hybrid retrieval in a conversational tourism recommender, thereby addressing the central problem of mapping natural-language requests to retrieval operations over recommendation

data [TB25]. TRACE extends this line of work toward sustainable tourism by combining conversation with agentic counterfactual explanations, which shows that agentic designs can also be used to justify and gently steer recommendation choices rather than only generate them [Ban+26].

At the same time, these developments are not specific to tourism. Gao et al. describe conversational recommender systems more generally as an interaction setting in which preference elicitation, recommendation, and dialogue management must be coordinated jointly [Gao+21]. Earlier work by Sun and Zhang already formalized conversational recommendation as a multi-turn decision process, which provides an important conceptual bridge between classical interactive recommenders and newer agentic designs [SZ18]. More recent LLM-based work in e-commerce argues explicitly that conversational recommendation and language models are mutually reinforcing because LLMs can interpret richer user intents while the recommender system provides item grounding and task structure [Liu+23].

Another relevant line of work concerns hybrid interaction beyond the travel domain. Feng et al. describe a large-language-model-enhanced conversational recommender system in which the language model is embedded into a broader recommendation stack instead of acting as a standalone chatbot [Fen+23]. In parallel, research on latent linear critiquing shows that effective recommendation often combines dialogue-like feedback with structured comparison of alternatives, which is closely aligned with hybrid interaction ideas [Li+20]. More generally, hybrid graphical-conversational interface research argues that conversational and graphical elements should be combined because they support different but complementary user tasks [Jär25].

3.3 Positioning of This Thesis

The reviewed work suggests two complementary developments. On the one hand, classical travel recommenders emphasize explicit user control and inspectable interaction. On the other hand, recent agentic approaches address intent interpretation, retrieval, and recommendation generation in more flexible conversational settings. This thesis is positioned at the intersection of these directions.

More specifically, the thesis investigates a prototype that combines data-grounded conversational recommendation with a hybrid user interface that exposes recommendation outcomes in a form that users can inspect and refine. Based on the currently available source material, the explicit coupling of multi-turn conversational recommendation, agentic workflow decomposition, and a hybrid interaction layer appears to be a useful focal point for this work.

4 Recommendation Generation

This chapter introduces the recommendation generation component of the prototype through an incremental perspective. Instead of presenting the full recommendation workflow at once, the chapter is structured around the main building blocks that progressively establish a grounded and inspectable recommendation process. It first introduces the data foundation on which recommendation output is based. It then outlines the framework used to evaluate how the system reasons during recommendation generation.

4.1 Data Grounding for Recommendation Generation

The recommendation component is grounded in a destination catalogue that is based on the dataset used in Sarica's DestiRec thesis [Sar25]. In the current prototype, this catalogue acts as the primary structured source from which destinations can be filtered, compared, and ranked. This grounding constraint is important for the overall system design. The prototype can only recommend destinations that are explicitly represented in this catalogue, and it can only reason over preference dimensions that are encoded in the available attributes.

For the recommendation generation approach considered in this thesis, the dataset provides 163 usable destination regions that form the grounding basis of the system.

4.1.1 Schema and Value Encoding

The dataset combines identification fields, cost information, temporal suitability fields, and thematic descriptors. Table 4.1 summarizes the schema used for recommendation generation.

The table shows that the schema combines four different layers of information. The first layer identifies and localizes the destination region. The second layer captures a direct budget signal through `cost_per_week`. The third layer describes when a destination is suitable through the month-specific fields. The fourth layer characterizes what kind of travel experience a region is associated with through thematic descriptors such as nature, culture, or watersports. Together, these layers provide the basic structure on which later filtering and preference interpretation operate.

Field	Value type	Description
parent_region	text	Broad geographic grouping of a destination region
region	text	Name of the destination region
u_name	text code	Unique identifier of the destination region
cost_per_week	numeric value	Estimated weekly travel cost
popularity	ordinal scale	General popularity indicator of the destination region
jan to dec	ordinal scale	Month-specific suitability values from January to December
safety	ordinal scale	Safety-related characterization of the destination region
nature	ordinal scale	Strength of nature-oriented travel characteristics
hiking	ordinal scale	Strength of hiking-related travel characteristics
beach	ordinal scale	Strength of beach-related travel characteristics
watersports	ordinal scale	Strength of water-sport-related travel characteristics
entertainment	ordinal scale	Strength of entertainment-oriented travel characteristics
wintersports	ordinal scale	Strength of winter-sport-related travel characteristics
culture	ordinal scale	Strength of cultural travel characteristics
culinary	ordinal scale	Strength of food-related travel characteristics
architecture	ordinal scale	Strength of architecture-related travel characteristics
shopping	ordinal scale	Strength of shopping-related travel characteristics

Table 4.1: Schema overview of the recommendation dataset

Most temporal and thematic fields are not stored as raw continuous numbers. Instead, they use an *ordinal scale* with the symbols `-`, `-`, `o`, `+`, and `++`. In this context, an *ordinal scale* means that the values express an ordered progression from lower to higher suitability or strength, without claiming that the distance between neighboring levels is measured exactly. These symbols are mapped to the numeric values 0.0, 0.25, 0.5, 0.75, and 1.0. The same mapping is also applied to the `popularity` field. This allows the dataset to express relative strength and suitability in a comparable way across many different attributes, while still remaining coarse-grained rather than highly fine-grained.

4.1.2 Hard and Soft Data Categories

For recommendation generation, it is useful to distinguish between hard and soft data categories. This distinction reflects how different parts of the dataset should influence the recommendation process when they are matched against user input. In the following work, hard preferences will be referred to as *filters*, whereas soft preferences will be referred to as *preferences* or *user interests*.

Hard data categories are those that define the feasible search space more directly. In the present dataset, these include geographic scope through `parent_region`, budget through `cost_per_week`, and seasonality through the monthly suitability fields from `jan` to `dec`. These categories are treated as hard because users often formulate them as actual constraints. If a user asks for destinations in Europe, for a limited budget, or suitable for a specific month or season, violating such conditions usually makes a recommendation substantially less useful. These attributes therefore function naturally as filters that narrow the candidate set before finer trade-offs are considered.

Soft data categories are those that describe thematic interests rather than strict feasibility constraints. In the present dataset, these include `nature`, `hiking`, `beach`, `watersports`, `entertainment`, `wintersports`, `culture`, `culinary`, `architecture`, and `shopping`. These attributes are better interpreted as signals of what kind of travel experience the user is looking for. In many realistic travel queries, such interests are not absolute requirements. A destination can still be useful even if it only partially matches them. For that reason, these categories are more suitable for ranking, weighting, and preference balancing than for strict exclusion.

This division is important for both recommendation quality and interpretability. If all attributes were treated as equally strict, the system would risk eliminating plausible destinations too early. If all attributes were treated as equally soft, the system could return destinations that violate explicit user constraints on geography, budget, or time of travel. Separating hard and soft categories therefore supports a more faithful mapping from natural language requests to recommendation logic. Filters delimit what is acceptable at all, whereas preferences or user interests determine which of the

acceptable candidates is most aligned with the user's interests.

4.1.3 Geographic Coverage and Granularity

The dataset is distributed across Europe, North America, South America, Africa, Asia, Australia, and Antarctica. More importantly, its internal structure follows travel characteristics rather than political borders alone. The regional division is designed to represent coherent travel spaces: areas with similar climate, landscape, activities, seasonality, or cultural profile can be grouped into one recommendation unit, whereas large countries can be divided into several units when different parts of the same country differ substantially along these dimensions.

World region	Records
Europe	32
North America	34
South America	17
Africa	21
Asia	50
Australia	8
Antarctica	1
Total	163

Table 4.2: Operational destination records by world region

This means that the recommendation units are intentionally not uniform at the country level. Some correspond roughly to one country, some combine several neighboring areas, and some subdivide large countries into multiple regions. They should therefore be interpreted primarily as travel regions rather than as political entities. This structure is important for recommendation generation because users usually search for a particular kind of travel experience, and the dataset is organized to express these experiential differences more clearly than a strictly country-based representation would.

4.1.4 Distribution of Costs and Seasonal Signals

The cost field provides one of the clearest structured anchors for recommendation generation. Table 4.3 summarizes the main distribution statistics for the weekly cost values, both for the full dataset and across the major continental groups. Overall, the cost structure is centered in the middle range, with a mean of 485.25 EUR, a 50th percentile of 400 EUR, and a 75th percentile of 560 EUR. The gap between the mean

and the median indicates that the distribution is pulled upward by a smaller number of more expensive regions.

Region group	Min	Max	Mean	P50	P75
Overall	250	4500	485.25	400	560
Europe	350	750	402.50	350	420
North America	250	975	564.41	600	693.75
South America	310	625	472.06	400	625
Africa	350	560	415.00	390	450
Asia	250	560	411.30	400	450
Australia	490	800	652.50	655	750
Antarctica	4500	4500	4500.00	4500	4500

Table 4.3: Weekly cost statistics overall and across region groups

Across continents, the distribution is not uniform. Europe, Africa, and Asia are comparatively concentrated in the lower to middle cost range, with means close to 400 EUR and 50th percentiles between 350 EUR and 400 EUR. North America and Australia are clearly more expensive on average, with North America centered around 600 EUR and Australia even higher. Antarctica is an extreme outlier with a single very high-cost region, so it should be interpreted separately from the rest of the dataset rather than as part of the ordinary travel budget distribution.

The seasonal profile is also informative. When the monthly suitability values are mapped to their numeric equivalents and averaged across all records, the strongest months are October and September, followed closely by May and June. The weakest months are January, December, and February. This suggests that the dataset is structurally strongest for spring, summer, and early autumn travel.

4.1.5 Thematic Profile

The thematic profile of the dataset is defined through the soft categories `nature`, `hiking`, `beach`, `watersports`, `entertainment`, `wintersports`, `culture`, `culinary`, `architecture`, and `shopping`. These categories describe different thematic aspects of a destination region and characterize the kinds of experiences associated with it.

Taken together, these categories cover several complementary dimensions of travel. Some fields focus on outdoor and landscape-oriented experiences, especially `nature`, `hiking`, and `wintersports`. Others capture coastal and water-oriented travel through `beach` and `watersports`. A third group reflects urban and cultural experiences through `culture`, `architecture`, and `entertainment`. Finally, `culinary` and `shopping` represent

forms of destination appeal that are tied less to geography alone and more to local atmosphere, consumption, and city or regional lifestyle.

The categories therefore span a broad range of travel themes, from nature-oriented and activity-oriented travel to urban, cultural, and lifestyle-oriented travel. They describe destination regions not through one single theme, but through a combination of thematic dimensions that can coexist within the same region.

At the same time, the categories are intentionally broad. They do not attempt to represent every niche activity or every subtle distinction between destinations. Instead, they provide a compact thematic characterization of regions at a level that remains general enough to be applied consistently across the full dataset.

4.2 Technology Used for Recommendation Generation

4.2.1 Framework Basis

The recommendation workflow in the developed system is implemented in Python through a combination of the LangChain and LangGraph frameworks. LangChain provides the higher-level components for building language-model-based application logic, especially the integration of models, prompts, and tool-oriented agent behavior [Lan26a]. LangGraph provides the lower-level orchestration layer for building stateful agent workflows as explicit graphs with controlled transitions between processing steps [Lan26b]. In the present system, LangChain is therefore used mainly for agent construction, while LangGraph is used mainly for execution structure and control flow.

4.2.2 Graph-Based Workflow Concepts

In the context of this system, a *graph* is the overall recommendation workflow represented as a directed processing structure. It defines which steps exist, how state moves between them, and under which conditions the workflow branches, continues, or terminates. A *node* is one executable step inside this graph. Nodes perform concrete tasks such as loading conversation state, routing the request, synthesizing the user request, extracting filters, generating recommendations, or producing the final response. An *agent* is the language-model-based decision component used within some of these nodes. In this system, agents are responsible for tasks that require interpretation or generation, such as request routing, filter extraction, synthesized-query construction, and response generation. In this sense, the agent does not represent the whole application. Rather, it is one reasoning component embedded within a broader graph-based workflow.

4.2.3 Storage and Data Access

The destination data used during recommendation generation is stored in a SQL database. This is used as a functional storage engine that keeps the travel destination data in one location and allows access through standard SQL queries over the destination table and its structured attributes.

4.3 System Reasoning Evaluation Framework

The system reasoning evaluation framework considered here focuses on the single-turn setting. Its purpose is to understand how a given recommendation architecture handles extraction and reasoning for single-shot travel queries. In this context, the term *filter* refers to the hard categories defined in Subsection 4.1.2, namely geographic scope, budget, and seasonality. The term *preference* or *user interest* refers to the soft thematic categories introduced in the same subsection.

Methodologically, the framework varies these two dimensions independently. One axis controls how clearly filters are expressed in a query. The other axis controls how specifically preferences are formulated. This makes it possible to observe whether a system architecture can identify filters and preferences under increasingly explicit or increasingly narrow query formulations, while the overall task remains the same.

The field symbols used in the framework are summarized in Table 4.4. The three filter levels range from no explicit filter signal to explicit machine-checkable constraints. The three preference levels range from broad thematic interests to highly specialized travel intents. In addition, the evaluation contains a separate unrelated-query category for inputs that do not belong to the travel recommendation task at all.

The full evaluation matrix is obtained by combining one filter level with one preference level. This results in nine travel-related cells, shown in Table 4.5. A query in `F0_P0` is open and weakly structured because it contains neither explicit filters nor a narrowly defined interest. A query in `F1_P1` combines weak filter hints with a more recognizable experience. A query in `F2_P2` combines explicit filters with a highly specialized travel intent. The matrix therefore creates controlled variation in single-turn reasoning difficulty while keeping the task domain constant.

To instantiate this matrix, a synthetic single-turn evaluation set was generated with GPT-5.4. The generation process followed a dedicated prompt specification documented in `backend/src/evaluation/test_data_generation_plan.md`. The same matrix is used across the compared system versions, each matrix cell contains 100 entries, and a separate set of 100 unrelated queries is kept outside the main matrix. These unrelated queries are included to assess how the system handles inputs that are outside its intended scope of purpose, namely cases that should not be processed

Symbol	Name	Description	Example
F0	No Filter	No explicit continent, budget, or season is stated.	"I want to travel somewhere nice."
F1	Vague Filter	A filter signal is present, but only indirectly or weakly expressed.	"without spending a fortune"
F2	Explicit Filter	Continent, budget, season, or several of them are stated in a machine-checkable form.	"Europe this summer for under $1500"
P0	Very Vague Preference	Only a broad interest is stated, such as beach, food, mountains, or city travel.	"Looking for a big city where I can walk around all day."
P1	Experience-Focused Preference	The query points to a recognizable travel experience without becoming highly narrow.	"Point me to a place where I can just drink amazing wine by a vineyard all day."
P2	Highly Specialized Preference	The intended experience is narrow and tied to a specific activity, landmark, place, food, or object.	"Where is the best spot to go scuba diving with manta rays?"
U	Unrelated Query	The input is outside the system's intended travel recommendation scope and should not be interpreted as a normal recommendation request.	"How do I reset my laptop password?"

Table 4.4: Interpretation of the single-turn matrix symbols

	P0 Very Vague Preference	P1 Experience-Focused Preference	P2 Highly Specialized Preference
F0 No Filter	broad interest without explicit filters	concrete experience without explicit filters	narrow interest without explicit filters
F1 Vague Filter	vague filter with broad interest	vague filter with concrete experience	vague filter with highly specialized interest
F2 Explicit Filter	explicit filter with broad interest	explicit filter with concrete experience	explicit filter with highly specialized interest

Table 4.5: Single-turn reasoning evaluation matrix

as travel recommendation requests at all. As a result, the single-turn set contains 900 travel-related evaluation queries and an additional 100 unrelated queries. The queries were intentionally written in varied and non-templated language so that the evaluation captures reasoning over diverse formulations rather than recognition of repeated phrasing.

This evaluation data is intended to provide a better understanding of how a given system architecture reasons about a single user request. It does not aim to validate the final recommendation list as such, nor to judge the overall quality of the returned recommendations. Instead, the focus is on whether the system can identify and extract active filters and active user interests from one query, and on how clearly this reasoning becomes observable in different architectural setups. In this sense, the framework is aimed at reasoning analysis for single-shot queries rather than at end-to-end recommendation validation.

The same evaluation data is run across different recommendation architectures with different capabilities so that reasoning behavior can be compared under identical input conditions. This includes architectures that expose little internal structure as well as architectures that produce more explicit intermediate reasoning and extraction steps. In addition, the evaluation is executed with both a locally runnable open-source model via Ollama, namely Llama 3.1 with 8B parameters, and the commercial model GPT-4o. These two models were chosen in order to compare a smaller local open-source model with a commercial model, to make the evaluation results more objective, and to avoid limiting the analysis to a single underlying model.

4.4 Recommender V1.0

Recommender V1.0 is the earliest graph-based recommendation workflow used in the system. It is intentionally simple and consists of one agent and one database-query

tool embedded in a small iterative LangGraph workflow. The purpose of this version is to let the agent interpret the user request, turn this interpretation into a structured query over the destination data, inspect the returned candidate regions, and then either continue the search by refining the query or finish with a response.

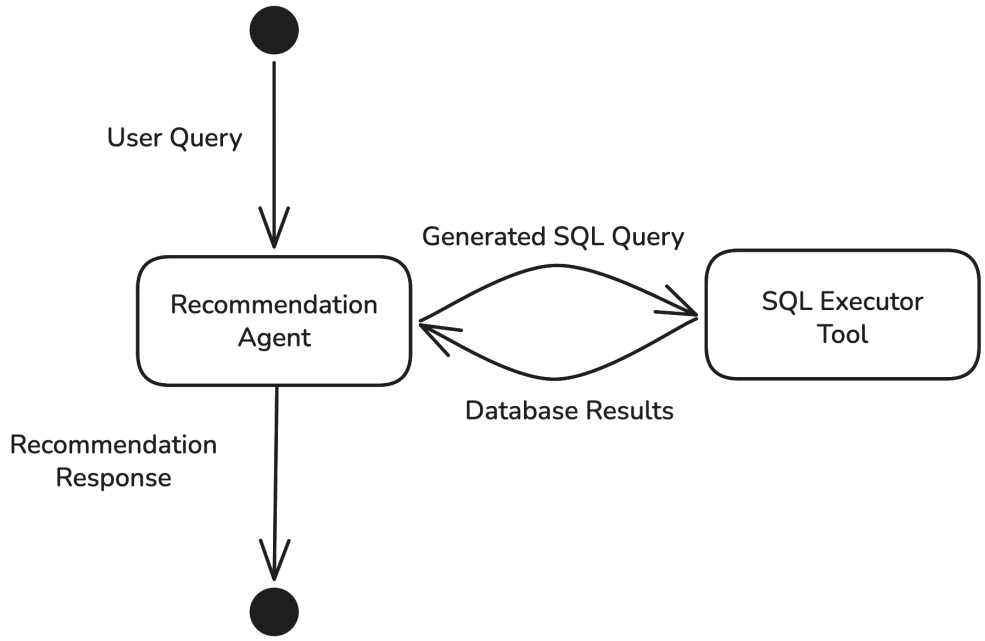


Figure 4.1: System diagram of Recommender V1.0

At the graph level, the workflow is centered on an agent-tool loop. The recommendation agent node receives the current user request and decides whether to request one tool execution or to finish the run with a final response and a list of recommendations. If a tool request is produced, execution is routed to the SQL tool node. The tool result is then passed back to the recommendation-agent node, so that the agent can either issue another query or finish the run.

The remainder of the architecture is deliberately minimal. The agent has access to one read-only retrieval tool, `query_travel_destinations_sql`, which operates over the `travel_destinations` table. Recommendation generation is therefore centered on one core operation: translating the interpreted user request into explicit SQL-based retrieval over structured destination attributes.

The prompting strategy in V1.0 follows a ReAct-like pattern. The prompt frames the model as a general-purpose agent that must alternate between two actions: request one tool step or finish with a final recommendation response. It explicitly describes the

available destination-table fields, states that tool results are the only admissible evidence for final recommendations, and instructs the agent to use the latest user request as the main trigger. It also encourages an iterative search strategy: first formulate the travel intent, then retrieve supporting evidence, and finally respond only when the available candidates are sufficient, as formalized in the V1.0 prompt implementation [Pro26a].

In practice, this means that V1.0 handles both filters and preferences through query construction rather than through separate extraction stages. The architecture exposes its reasoning mainly through the SQL query requested by the agent and through the sequence of tool decisions that follow from it. For that reason, V1.0 serves as a compact baseline for examining how far a single agent with one structured retrieval tool can carry the recommendation process.

4.4.1 Results on the Synthetic Single-Turn Queries

The synthetic single-turn evaluation set [Pro26b] was run for Recommender V1.0 with two underlying models: Llama 3.1 8B and GPT-4o. For this recommender, the most immediate comparison criterion is the rate of technically successful runs, because the architecture depends on explicit tool requests and explicit query construction. A run is therefore counted as successful when the workflow finishes without execution error, regardless of whether it returns recommendations or ends with an empty result. The overall comparison is summarized in Table 4.6.

Model	Queries	Successful runs	Error runs	Success rate
Llama 3.1 8B	1000	703	297	70.3%
GPT-4o	1000	998	2	99.8%

Table 4.6: Overall execution success of Recommender V1.0 on the synthetic single-turn queries

The contrast is substantial. GPT-4o remains almost fully stable across the synthetic evaluation, whereas the Llama-based variant fails in nearly one third of all runs. This difference indicates that the main weakness of V1.0 is not only the retrieval design itself, but also the stability of the model when it must repeatedly formulate valid tool requests and valid structured queries.

The error traces show that these failures are closely tied to SQL-query assembly and tool-use control. In the Llama-based runs, the dominant failure modes are missing SQL queries in tool requests and attempts to request another tool step in a final round where no further tool action is allowed. In addition, there are repeated SQL errors caused by invalid field references, malformed SQL syntax, and invalid value typing inside SQL

conditions. The GPT-4o runs show only two recorded failures, one SQL syntax failure and one SQL error caused by an invalid column reference.

Error category	Llama 3.1 8B	GPT-4o
Missing `sql_query` in tool request	121	0
Tool request issued in final round	121	0
Invalid SQL column reference	39	1
SQL syntax error	11	1
Invalid SQL value or type usage	3	0
Invalid SQL operator or function usage	1	0
Missing final response after finish	1	0
Total	297	2

Table 4.7: Observed V1.0 execution errors by category

These results suggest that the V1.0 architecture is highly sensitive to the model's ability to follow the prompt protocol and to formulate structurally valid SQL-oriented actions. For this reason, the execution-success comparison is already informative in itself. Before interpreting how well filters or preferences are captured, it is necessary to establish whether the architecture can reliably sustain the agent-tool loop without breaking its own query-construction logic. In the case of V1.0, this requirement is satisfied much more consistently by GPT-4o than by the smaller Llama model.

For the travel-related matrix queries, the result traces can be examined more directly through the SQL behavior that the architecture exposes. In this case, the analysis is necessarily approximate, because V1.0 does not externalize separate filter-extraction or preference-extraction structures. Instead, both become visible only through the SQL query requested by the agent. The counts below are therefore derived from successful runs only and were assigned manually by expert inspection of the result traces. A run is counted as having produced recommendations when it finishes with a non-empty recommendation list. A filter is counted as correctly identified when the SQL query preserves the expected hard constraints of the request. A preference is counted as mapped when the SQL query contains a visible proxy for the intended travel interest, for example through thematic fields such as `beach`, `nature`, `culinary`, or `culture`, or through a direct region-oriented substitute. This should be understood as an operational approximation of the architecture's observable reasoning rather than as a semantic judgment of recommendation quality.

These tables show a clear structural difference between the two model variants. GPT-4o is not only more stable overall, but also more consistent in translating the hard constraints of the query into SQL clauses. This becomes especially visible in the

Cell	Success	Recs. returned	Filters correct	Preference mapped
F0_P0	90	59	73	88
F0_P1	78	46	62	78
F0_P2	76	21	31	76
F1_P0	100	98	44	91
F1_P1	76	58	52	74
F1_P2	68	27	20	63
F2_P0	72	54	70	72
F2_P1	44	24	30	44
F2_P2	60	40	46	60

Table 4.8: Approximate matrix-level behavior of Recommender V1.0 with Llama 3.1 8B

Cell	Success	Recs. returned	Filters correct	Preference mapped
F0_P0	100	96	87	92
F0_P1	100	87	80	93
F0_P2	99	19	66	58
F1_P0	100	95	52	91
F1_P1	100	96	69	98
F1_P2	100	30	20	68
F2_P0	100	95	99	99
F2_P1	99	89	95	95
F2_P2	100	64	94	98

Table 4.9: Approximate matrix-level behavior of Recommender V1.0 with GPT-4o

explicit-filter cases `F2_P0`, `F2_P1`, and `F2_P2`, where correct filter handling is high and incorrect filter insertion is rare. The Llama-based variant, by contrast, shows a much weaker pattern in the same cells, even when the run itself completes successfully. This suggests that successful execution alone is not enough to guarantee coherent filter translation.

Measure	Llama 3.1 8B	GPT-4o
Recommendation rate	64.3%	74.7%
Filter-correct rate	64.5%	73.7%
Preference-mapped rate	83.6%	88.2%

Table 4.10: Aggregate matrix-level rates of Recommender V1.0 across successful runs

At the aggregate level, GPT-4o produces recommendations more often and preserves the expected hard constraints more often. The difference in preference mapping is smaller. This suggests that even the smaller model often finds some structured proxy for the user interest, but is less reliable in translating the full set of hard constraints into coherent SQL.

The preference column should be interpreted more cautiously. Because V1.0 represents preferences only through SQL query construction, the presence of a thematic mapping mainly shows that the architecture found a structured proxy for the user interest. In broad cases such as `P0`, this often works well because the request can be mapped to fields such as `beach`, `nature`, `hiking`, or `culinary`. In more specific cases, especially `P2`, the preference is often reduced to a broader proxy or to a literal region match. The tables therefore do not show whether the full nuance of the preference was preserved, but they do show whether the architecture was able to turn the request into some structured representation at all.

Model	Queries	Successful runs	Valid scope responses
Llama 3.1 8B	100	39	24
GPT-4o	100	100	100

Table 4.11: Successful-run rates of Recommender V1.0 on unrelated queries

The unrelated-query set should be interpreted separately because it does not test destination reasoning, but scope handling. In this context, a valid scope response means that the system finishes successfully and returns no recommendations for an out-of-scope input. Here the contrast is even sharper. GPT-4o completes all 100 unrelated-query runs without execution error and produces a valid scope response in all cases. The Llama-based variant completes only 39 of 100 successfully, and only 24

of these runs end with a valid scope response without recommendations. This means that the weaker model is not only less stable on travel-related reasoning, but also less reliable when the architecture is forced to reject or deflect requests that are outside the intended recommendation scope.

Beyond execution success, the result traces also show how V1.0 captures filters. In this architecture, filters are not extracted into dedicated intermediate structures. Instead, they become visible through the SQL clauses requested by the agent. When the user provides no explicit filters, as in `F0`, GPT-4o usually avoids hard geographic or budget constraints and formulates mainly thematic SQL conditions such as `beach > 0.8` or `nature > 0.7 AND hiking > 0.7`. This suggests that the architecture can keep broad preferences in view without unnecessarily inventing explicit filters. In `F1`, vague filters are often normalized into approximate SQL constraints, for example by mapping "without spending a fortune" to a low cost threshold or by turning "warmer months" into summer-month conditions. In `F2`, the GPT-based variant usually translates explicit region, budget, and season constraints directly into SQL, for example through `parent_region = 'Europe'`, `cost_per_week <= ...`, and month-based season clauses.

The Llama-based variant follows the same general strategy, but with noticeably lower reliability. For broad queries it often produces the expected thematic SQL restriction. However, for more explicit or more complex requests it more frequently introduces wrong month logic, incomplete query clauses, or syntactically invalid SQL. The high number of missing-`sql_query` and final-round tool-request errors shows that filter handling in this variant breaks down not only at the level of semantic interpretation, but already at the level of tool-control discipline.

Preference capture in V1.0 follows the same SQL-centered logic. Broad preferences from `P0` are usually mapped to one or two thematic fields, for example `beach`, `nature`, `hiking`, or `culinary`. This works relatively naturally because the dataset already offers direct schema fields for such themes. In `P1`, the architecture still captures the request, but often only through proxies. Ancient ruins may be translated into `culture` or `architecture`, a wine-region request into `culinary`, and an indie music scene into `entertainment`. This means that the broader experience is often retained, but only after being compressed into the limited field vocabulary of the destination table.

The strongest limitation appears in `P2`. Highly specialized requests are often flattened into either broad thematic proxies or literal region-matching attempts. In the GPT-based runs this can still produce coherent SQL, but the resulting query may already lose the original anchor by reducing it to culture, watersports, or one exact region name. In the Llama-based runs, the same type of specialization is more likely to produce malformed SQL or unstable query refinement. As a result, V1.0 can capture broad and moderately specific preferences to a limited degree, but its handling of narrow, anchor-specific user intents remains structurally weak because the architecture forces these intents into

direct SQL retrieval over a relatively small set of structured fields.

4.4.2 Limitations of Recommender V1.0

The analysis of Recommender V1.0 shows several structural limitations. The first limitation is architectural fragility. Because the recommender depends on explicit tool requests and explicit SQL construction, its behavior is highly sensitive to whether the underlying model can follow the tool protocol and produce valid structured queries. The observed runtime failures, missing SQL requests, and malformed SQL statements show that this dependency is substantial, especially for the smaller model.

The second limitation concerns how filters and preferences are represented internally. In V1.0, there is no dedicated intermediate layer for extracting or storing filters and no separate representation for user preferences. Instead, both are compressed directly into SQL constraints. This makes the architecture compact, but it also means that interpretation and retrieval are tightly coupled. As soon as the interpretation is imperfect, the retrieval logic is affected immediately, with little opportunity for later correction.

The third and more fundamental limitation concerns the treatment of user preferences expressed in natural language. In practice, V1.0 often flattens a user request into one or a few broad schema fields such as `culture`, `culinary`, `nature`, or `watersports`. This defeats an important part of the purpose of natural, unrestricted language input. Although the user can formulate a request in a nuanced and open-ended way, the architecture ultimately reduces this request to a limited set of predefined categories that may not be expressive enough for the intended travel experience. This limitation is not caused by the architecture alone. It is also related to the current dataset, whose present structure may not be sufficient to represent the full variety of user intents in a more fine-grained way. As a result, broad requests can still be handled to some extent, but more specific user intents are easily compressed into coarse proxies that do not preserve the full meaning of the original query.

This problem becomes most visible for highly specialized requests. If the intended experience is tied to a concrete landmark, route, activity, food, or place, V1.0 often has no direct representational mechanism for preserving that specificity beyond a literal region match or a thematic approximation. The architecture can therefore appear to handle natural-language input, while in fact translating it into a much narrower internal vocabulary. This creates a mismatch between the flexibility of the input channel and the limited expressiveness of the retrieval representation.

4.5 Embedding-Based Preference Matching

The third limitation of Recommender V1.0 identified in Subsection 4.4.2 – the compression of natural language preferences into a fixed set of schema fields – is not a surface-level issue that better prompting or more reliable tool use could resolve. It follows from the reliance on a predefined category vocabulary as the only bridge between the user query and the destination data. As long as every travel interest must be mapped to one of the ten soft categories, the system cannot preserve the semantic specificity of the original query regardless of how well the rest of the pipeline functions.

To mitigate this limitation, the next architectural version replaces category-based preference mapping with semantic similarity matching through embeddings. As introduced in Section 2.3, an embedding model maps texts to dense vector representations in a high-dimensional space, where semantically related inputs are positioned close to one another. Instead of translating a query into a schema field such as `culture` or `architecture`, the embedding-based approach compares the semantic content of the user request directly against descriptive representations of each destination. This preserves the specificity of the original query because the matching operates in a continuous semantic space rather than through a discrete set of category labels.

This shift has two important consequences. First, it decouples preference interpretation from the fixed category structure of the dataset. The embedding space can represent a much broader range of travel intents, including those that combine multiple themes, refer to specific activities or landmarks, or express experiential qualities that are difficult to encode as ordinal scales. Second, the similarity signal operates over descriptive destination content rather than over manually assigned category scores, so the expressiveness of the matching step depends on the richness of the destination representations rather than on the granularity of a predefined schema.

At the same time, embeddings do not replace the structured filtering layer. Hard constraints such as budget, geographic scope, and seasonality remain better handled through explicit SQL conditions because they require exact threshold comparisons rather than approximate similarity. The embedding-based matching is therefore introduced as a complementary component: filters narrow the candidate set deterministically, and semantic similarity ranks the remaining candidates according to how well their descriptive profiles align with the user's natural language request. This separation follows the same hard-soft distinction established in Subsection 4.1.2, but replaces the coarse category approximation for soft preferences with a continuous similarity signal.

The architecture that realizes this approach is presented in the following chapter as Recommender V2.0, where the embedding-based matching is integrated into a graph-based workflow alongside explicit filter extraction and structured retrieval.

5 System Architecture

This chapter outlines the architecture of the prototype travel destination recommendation system developed in this thesis. The system is organized around three main concerns: a backend that performs data access and recommendation orchestration, a frontend that combines conversational and graphical interaction, and a communication layer that connects both sides while supporting responsive feedback.

5.1 Overview

At a high level, the system accepts natural language travel requests, interprets them into actionable constraints, retrieves suitable destination candidates, and presents the results through a hybrid interface. The architecture is designed to support iterative refinement. Instead of treating each request as an isolated query, the system maintains conversational context so that later turns can narrow, revise, or extend earlier preferences.

The architecture also reflects the central design assumption of this thesis: recommendation quality depends not only on the final destination list, but also on whether the system can keep recommendation generation grounded in data and make the interaction understandable to the user.

5.2 Backend

The backend is responsible for managing recommendation logic, conversational state, data access, and language-model-based processing. It acts as the coordination layer between user requests and the underlying travel destination data.

5.2.1 Technologies Overview

The backend is implemented as a service-oriented layer that exposes recommendation functionality through an application programming interface. Its main responsibilities are request handling, orchestration of language-model interactions, execution of retrieval operations, and persistence of conversational context.

5.2.2 Data

The quality of the recommendation pipeline depends directly on the quality and structure of the destination data. Because user requests can mix explicit constraints with qualitative preferences, the data layer must support both structured filtering and richer textual retrieval.

Base Dataset and Its Limitations

The initial dataset provides the core set of travel destinations used by the prototype. However, a base dataset alone may be insufficient for natural language recommendation if it lacks descriptive fields, normalized attributes, or coverage of user-relevant aspects such as seasonal suitability, activity profiles, or budget-related characteristics.

Requirements for Dataset Extension

To support natural language queries, the dataset should be extended in a way that captures concepts users actually mention in conversation. This includes not only factual attributes but also descriptive representations that make semantic retrieval possible. The extension process should therefore improve attribute completeness, textual richness, and consistency of destination descriptors.

Interaction with the Storage Layer and Agentic Tool Exposure

The storage layer serves as the authoritative source of destination data and is accessed through controlled retrieval operations. In an agentic setup, these operations can be exposed as tools so that recommendation components retrieve concrete records instead of relying on unsupported generation. This design helps maintain grounding while still allowing flexible interpretation of user requests.

5.2.3 Embeddings and Language Model Interactions

Embeddings are used to represent textual information in a form suitable for semantic retrieval. Within the prototype, they complement direct attribute-based filtering by enabling approximate matching between user requests and destination descriptions. Language-model interactions are then used to interpret the request, derive retrieval cues, and synthesize the final response from retrieved evidence.

This separation is important. Embeddings support retrieval over descriptive similarity, whereas language models support interpretation and response construction. Combining

both can improve flexibility, but it also requires careful orchestration so that response generation remains tied to retrieved data.

5.2.4 Recommender

The recommender is the central decision-making component of the system. It translates user intent into candidate selection and ranking procedures while preserving enough structure to support explanation and refinement.

Design Objectives

The recommender is designed around four objectives: grounding recommendations in destination data, supporting iterative conversational refinement, separating hard constraints from softer preferences where possible, and producing outputs that can be surfaced clearly in the hybrid interface.

Baseline Recommendation Approaches

Two simpler baselines are considered in order to contextualize the proposed multi-agent workflow.

Single-Agent Recommendation with Full Prompt Context. In the first baseline, the entire conversational context and available destination information are provided directly to a single language model prompt. This design is simple, but it provides limited control over how constraints are extracted, retrieved, and applied.

Single-Agent Recommendation with Tool-Based Retrieval and Few-Shot Prompting. The second baseline still uses a single recommendation agent, but it supplements generation with tool-based retrieval from the structured data layer and with few-shot prompting. This design introduces grounding, yet it does not impose an explicit internal workflow for separating interpretation, retrieval, and response construction.

Multi-Agent Recommendation Framework

The main contribution of the prototype is a multi-agent recommendation framework in which specialized steps are separated into distinct responsibilities.

Context-Aware Interpretation of User Requests. The first step interprets the user's current message in light of the existing conversation. This includes identifying whether the user introduces new constraints, relaxes earlier preferences, requests clarification, or asks for a reformulation of existing recommendations.

User Preference Extraction and Constraint Derivation. After interpretation, the system derives structured recommendation constraints from the request. These may include regional preferences, seasonal preferences, and budget-related preferences. Distinguishing between explicit constraints and softer indications of interest is important because both categories should influence recommendation generation differently.

Aggregation of Travel Constraints into a Unified Recommendation Context. The extracted information is merged with previously stored conversational state. This aggregation step creates a unified recommendation context that can be reused in subsequent turns and reduces the risk that relevant earlier information is ignored.

Hybrid Retrieval and Candidate Recommendation Generation. Candidate generation combines semantic retrieval based on embeddings, direct keyword or text-based retrieval, and structured filtering over destination attributes. The resulting candidates are then ranked in a way that reflects both hard constraints and softer preference alignment.

Recommendation Enrichment and Response Construction. After candidate generation, the recommendation output is enriched with explanatory content that can be displayed in the user interface. This step should preserve the distinction between retrieved evidence and generated phrasing so that the final response remains both readable and grounded.

Conversational Memory Persistence and Recommendation Refinement. The final step stores the updated recommendation context for future turns. This allows users to continue refining the recommendation without restating the full query and makes multi-turn recommendation quality a first-class system objective.

5.2.5 Exposed API

The backend exposes its functionality through an application programming interface that separates state management from user interaction.

User Session. Session-related endpoints create and maintain the conversational scope within which user preferences and recommendation history are stored.

Travel Destinations. Destination-related endpoints provide controlled access to the underlying travel data and support retrieval operations required by the recommendation workflow.

Recommendation. Recommendation-related endpoints trigger the recommendation pipeline, return results, and provide incremental status information to the frontend where required.

5.3 Frontend

The frontend is the user-facing part of the prototype. Its purpose is not only to display final recommendations, but also to support the interaction pattern through which users formulate, inspect, and refine travel preferences.

5.3.1 Technologies Overview

The frontend is implemented as a web-based client that integrates conversational and graphical views into a single interaction flow.

5.3.2 User Interface Design

The user interface is designed around the principle that recommendation should remain inspectable. Users should be able to express preferences conversationally, but they should also receive structured visual feedback that helps them understand the recommendation outcome and continue the interaction productively.

5.3.3 Conversational Interaction Layer

The conversational layer captures natural language input and presents system responses in dialogue form. Its main role is to support nuanced preference expression, follow-up questions, and iterative refinement of travel constraints across multiple turns.

5.3.4 Graphical Interaction Layer

The graphical layer complements the conversation with structured visual elements. In the context of this thesis, this includes map-based or region-based exploration and other

interface components that help users compare alternatives or inspect recommendation outputs beyond plain text.

5.3.5 Recommendation Presentation

Recommendation presentation combines textual explanation with structured destination information. This part of the interface is critical because it connects backend recommendation logic with user understanding. A recommendation is only actionable if users can quickly see which destinations were suggested, how they relate to stated preferences, and what can be refined next.

5.4 Communication Between Components

The backend and frontend interact through a communication model that must support both correctness and responsiveness.

5.4.1 HTTP Request-Response Communication

Conventional request-response communication is suitable for many operations in the system, including session creation, retrieval of destination data, and submission of recommendation requests. It provides a simple and well-understood mechanism for integrating the frontend with backend services.

5.4.2 Limitations of Basic Request-Response Interaction

For recommendation retrieval, a purely blocking request-response model can be limiting because the user receives feedback only after the entire processing pipeline is complete. In a multi-step recommendation workflow, this can obscure progress and make waiting times feel less predictable, especially when several retrieval and generation operations are involved.

5.4.3 Server-Sent Events for Incremental Feedback

To address this limitation, the system can use server-sent events to stream intermediate status updates from the backend to the frontend. This does not change the recommendation logic itself, but it improves the interaction quality by making long-running processing steps more visible and by helping users understand that the system is actively progressing through the request.

6 Evaluation

This chapter defines the evaluation design used to assess the proposed system. The evaluation is divided into two complementary parts. The first part focuses on the internal recommendation workflow through a synthetic benchmark. The second part evaluates the user experience of the multi-agent prototype through a survey-based study.

6.1 Synthetic Evaluation

The synthetic evaluation focuses solely on the recommendation flow. Its purpose is to compare simpler recommendation baselines with the proposed multi-agent framework under controlled conditions that make relevance and conversational consistency easier to inspect.

6.1.1 Synthetic Ground-Truth Dataset Construction

The synthetic benchmark is based on a constructed ground-truth dataset representing realistic travel recommendation scenarios. The intention is to define user personas with hard constraints and soft travel preferences, and to pair these personas with destination outcomes that serve as a reference for evaluation.

The benchmark should include both single-turn and multi-turn cases. Multi-turn scenarios are especially important because they make it possible to test whether the system correctly preserves earlier constraints when later messages modify only parts of the request.

6.1.2 Evaluation Framework: Recommendation Relevance

The relevance-oriented part of the evaluation compares the naive baseline approaches and the multi-agent recommendation framework on the same dialogue scenarios. The main question is whether the returned recommendations satisfy explicit constraints and align with softer stated preferences.

This comparison should make it possible to analyze not only final recommendation quality, but also how much the explicit workflow decomposition contributes relative to

simpler prompt-based designs.

6.1.3 Evaluation Framework: Context Persistence and Memory

The second synthetic evaluation perspective focuses on conversational memory. Here, the objective is to test whether earlier user constraints continue to influence later recommendation turns and whether the system avoids contradictions when preferences evolve over time.

This part of the evaluation is important because a travel recommendation assistant becomes substantially less useful if users must repeatedly restate previously established constraints.

6.2 User Survey-Based Evaluation

The second part of the evaluation examines the user experience of the multi-agent recommendation framework and the usability of the application interface. Unlike the synthetic benchmark, this part focuses on perceived interaction quality rather than controlled comparison against baseline systems.

The survey uses five-point Likert-scale items ranging from 1 (strongly disagree) to 5 (strongly agree). The planned questionnaire covers the following dimensions:

1. **Interface Adequacy:** "I found the application interface intuitive to use."
2. **Conversational Interaction Quality:** "The system accurately understood my travel requests and preferences."
3. **Conversational Flow:** "The conversation with the system felt natural and logically connected."
4. **Conversational Consistency:** "The system did not lose or misinterpret information that I provided during the conversation."
5. **Recommendation Relevance:** "The recommended destinations matched my interests and travel needs."
6. **Recommendation Usefulness:** "The recommendations helped me identify destinations I would consider visiting."
7. **Recommendation Transparency:** "The region descriptions helped me understand how the recommended regions matched my travel preferences."

8. **System Responsiveness:** "The system provided adequate feedback while processing my requests, and the waiting times were acceptable."
9. **Hybrid Interface Satisfaction:** "The combination of conversational interaction and map-based visualization and region descriptions enhanced my experience."
10. **Overall Satisfaction:** "Overall, I am satisfied with the travel destination recommendation application."

Taken together, these questions evaluate whether the proposed hybrid interaction design improves usability, transparency, and overall user satisfaction.

7 Conclusion

This thesis investigates how a hybrid user interface and an agentic recommendation workflow can be combined in a travel destination recommendation application. The work is motivated by the observation that travel preferences are often expressed most naturally in open-ended language, while recommendation usefulness still depends on data grounding, context preservation, and understandable presentation.

The final version of this chapter will return to the research objectives stated in Section 1.3, summarize the implemented system and evaluation findings, and discuss the main limitations of the proposed approach.

Abbreviations

List of Figures

List of Tables

Bibliography

[] *Destination Finder*. URL: https://destination-finder.netlify.app/.

[AGZ24] M. Aliannejadi, J. Gwizdka, and H. Zamani. *Interactions with Generative Information Retrieval Systems*. arXiv preprint arXiv:2407.11605. 2024.

[AH22] T. Alenezi and S. Hirtle. "Normalized Attraction Travel Personality Representation for Improving Travel Recommender Systems." In: *IEEE Access* 10 (2022), pp. 56493–56503.

[AT05] G. Adomavicius and A. Tuzhilin. "Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions." In: *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), pp. 734–749.

[Ban+26] A. Banerjee, A. Satish, W. W"orndl, and Y. Deldjoo. "TRACE: A Conversational Framework for Sustainable Tourism Recommendation with Agentic Counterfactual Explanations." In: *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2026. DOI: 10.1145/3805712.3808370.

[BMV14] J. Borràs, A. Moreno, and A. Valls. "Intelligent Tourism Recommender Systems: A Survey." In: *Expert Systems with Applications* 41.16 (2014), pp. 7370–7389. DOI: 10.1016/j.eswa.2014.06.007.

[Bur02] R. Burke. "Hybrid Recommender Systems: Survey and Experiments." In: *User Modeling and User-Adapted Interaction* 12.4 (2002), pp. 331–370.

[Cha24] H. Chase. *What Is an AI Agent?* June 28, 2024. URL: https://www.langchain.com/blog/what-is-an-agent.

[Che] T. Chen. *Agentic AI for Trip Planning Optimization Application*.

[CP12] L. Chen and P. Pu. "Critiquing-Based Recommenders: Survey and Emerging Trends." In: *User Modeling and User-Adapted Interaction* 22.1-2 (2012), pp. 125–150.

[CT20] K. Chaudhari and A. Thakkar. "A Comprehensive Survey on Travel Recommender Systems." In: *Archives of Computational Methods in Engineering* 27.5 (2020), pp. 1545–1571. DOI: 10.1007/s11831-019-09363-7.

[Die+22] L. W. Dietz, M. Sertkan, S. Myftija, S. Thimbiri Palage, J. Neidhardt, and W. Wörndl. "A Comparative Study of Data-Driven Models for Travel Destination Characterization." In: *Frontiers in Big Data* 5 (2022), p. 829939.

[Fen+23] Y. Feng, S. Liu, Z. Xue, Q. Cai, L. Hu, P. Jiang, K. Gai, and F. Sun. *A Large Language Model Enhanced Conversational Recommender System*. arXiv preprint arXiv:2308.06212. 2023. DOI: 10.48550/arXiv.2308.06212.

[Gao+21] C. Gao, W. Lei, X. He, M. de Rijke, and T.-S. Chua. "Advances and Challenges in Conversational Recommender Systems: A Survey." In: *AI Open* 2 (2021), pp. 100–126. DOI: 10.1016/j.aiopen.2021.06.002.

[Gao+23] Y. Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv preprint arXiv:2312.10997. 2023.

[Jan22] D. Jannach. "Evaluating Conversational Recommender Systems." In: *Artificial Intelligence Review* 56.3 (2022), pp. 2365–2400.

[Jär25] R. Järvinen. "Prompting the Future: Visual and Interaction Design Guidelines for Hybrid Graphical–Conversational and Generative AI User Interfaces." Bachelor's thesis. Aalto University, Dec. 2025.

[KR12] J. A. Konstan and J. Riedl. "Recommender Systems: From Algorithms to User Experience." In: *User Modeling and User-Adapted Interaction* 22.1-2 (2012), pp. 101–123.

[Lan26a] LangChain. *LangChain overview*. 2026. URL: https://python.langchain.com/docs/introduction/ (visited on 06/27/2026).

[Lan26b] LangChain. *LangGraph overview*. 2026. URL: https://docs.langchain.com/oss/python/langgraph/overview (visited on 06/27/2026).

[Li+20] H. Li, S. Sanner, K. Luo, and G. Wu. "A Ranking Optimization Approach to Latent Linear Critiquing for Conversational Recommender Systems." In: *Proceedings of the 14th ACM Conference on Recommender Systems*. 2020, pp. 13–22. DOI: 10.1145/3383313.3412240.

[Liu+23] Y. Liu, W. Zhang, Y. Chen, Y. Zhang, H. Bai, F. Feng, H. Cui, Y. Li, and W. Che. "Conversational Recommender System and Large Language Model Are Made for Each Other in E-commerce Pre-sales Dialogue." In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023, pp. 9587–9605. DOI: 10.18653/v1/2023.findings-emnlp.643.

[Lü+12] L. Lü, M. Medo, C. H. Yeung, Y.-C. Zhang, Z.-K. Zhang, and T. Zhou. "Recommender Systems." In: *Physics Reports* 519.1 (2012), pp. 1–49.

[Mik+13] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. "Distributed Representations of Words and Phrases and their Compositionality." In: *Advances in Neural Information Processing Systems 26*. 2013, pp. 3111–3119.

[Pen+25] Q. Peng, H. Liu, H. Huang, J. Yang, Q. Yang, and M. Shao. "A Survey on LLM-powered Agents for Recommender Systems." In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. 2025, pp. 11574–11583. DOI: 10.18653/v1/2025.findings-emnlp.620.

[Pro26a] Project repository. *Internal Recommender V1.0 Prompt Source*. Prompt implementation used in the developed system. % TODO: Add repository URL or stable file reference. 2026.

[Pro26b] Project repository. *Single-Turn Recommendation Evaluation Dataset*. 2026. URL: https://github.com/waldemar99zielinski/travel_recommender/tree/main/backend/src/evaluation/data (visited on 06/28/2026).

[RG19] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.

[Sar25] C. N. Sarica. "DestiRec: Investigating User Control to Influence Complex Recommender Systems." Examiner: Prof. Dr.-Ing. Jörg Ott. Supervisor: Dr. Wolfgang Wörndl. Thesis. Technische Universität München, Oct. 24, 2025.

[SK09] X. Su and T. M. Khoshgoftaar. "A Survey of Collaborative Filtering Techniques." In: *Advances in Artificial Intelligence* 2009 (2009), pp. 1–19.

[SZ18] Y. Sun and Y. Zhang. "Conversational Recommender System." In: *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 2018, pp. 235–244. DOI: 10.1145/3209978.3210002.

[TB25] A. Tandon and A. Banerjee. "Evaluating User Intent Classification and Hybrid Retrieval in a RAG-based Conversational Tourism Recommender System." In: *CEUR Workshop Proceedings*. Vol. 4052. 2025.

[Zha+26] W. X. Zhao et al. "A Survey of Large Language Models." In: *Frontiers of Computer Science* 20.12 (2026).